

IMATS ENGINEERING

Department of Computer Science & Engineering

ASSESSMENT TOOL 2- DEBUGGING & OPTIMISATION TASK

Course Code:	CSA12	Course Title:	Computer Architecture
CO Assessed:	CO2	Assessment:	ASSESSMENT TOOL2- DEBUGGING & OPTIMISATION TASK
Weightage:	20%	Total Marks:	25
CO2 Statement:	<i>Apply integer and floating-point arithmetic algorithms including Booth's multiplication, restoring/non-restoring division, and IEEE 754 standard operations to solve fixed-point and floating-point computational problems. (BL3)</i>		
Student Name:	N. Tharun Sai	Reg. No.:	192472382
Department:	CSE-AI	Date:	

ANNEXURE A

1. A program performing signed integer addition using 2's complement produces incorrect results when adding large values (e.g., +120 and +50 in 8-bit representation). Debug the issue by analyzing the binary computation and identifying whether overflow has occurred. Propose a solution to correctly detect and handle overflow in such cases.
2. A processor using a ripple carry adder shows significant delay in executing arithmetic operations in a real-time system. Debug the performance bottleneck by examining how carry propagates through each bit. Suggest an optimized solution using a carry look-ahead adder, and explain how it reduces delay.
3. A multiplication unit implementing Booth's algorithm gives incorrect results for certain negative operands. Debug the step-by-step execution of the algorithm, focusing on bit-pair checking, arithmetic shifts, and sign extension. Identify the possible source of error and suggest corrections to ensure accurate signed multiplication.
4. An embedded system using the restoring division algorithm experiences inefficiency due to repeated restoration steps, leading to increased execution time. Debug the algorithm's workflow and identify where unnecessary operations occur. Propose an optimized approach using the non-restoring division method, explaining how it improves performance.
5. A scientific application using IEEE 754 single precision floating-point representation produces inaccurate results when adding very small and very large numbers. Debug the issue by analyzing exponent alignment, normalization, and rounding errors. Suggest optimization techniques, such as using double precision or algorithmic adjustments, to improve accuracy.

1. Debugging Signed Integer Addition Using 2's Complement Overflow

Introduction

Signed integer addition in computers is performed using the 2's complement representation. This method allows both positive and negative numbers to be added using the same hardware. However, when the result exceeds the range of the available bits, an overflow occurs, producing an incorrect result.

Problem

A program performs signed 8-bit addition using 2's complement but gives incorrect results when adding **+120 and +50**.

Binary Representation

- $+120 = 01111000$
- $+50 = 00110010$

Binary Addition

```
01111000 (+120)
+ 00110010 (+50)
```

10101010

Result = **10101010**

Analysis

- Binary 10101010 is negative in 2's complement.
- Decimal value:
- Invert: 01010101
- Add 1: 01010110 = 86
- Therefore result = **-86**

Actual mathematical result:

$$120 + 50 = 170$$

An 8-bit signed integer can represent only **-128 to +127**.

Since **170** is outside this range, **overflow occurs**.

Overflow Detection

Overflow occurs when:

- Two positive numbers produce a negative result.
- Two negative numbers produce a positive result.

Another method:

- Compare the carry into the MSB and carry out of the MSB.
- If they are different, overflow has occurred.

Solution

- Check the overflow flag after every signed addition.
- If overflow occurs:
- Display an overflow error.
- Use a larger data type (16-bit or 32-bit integer).
- Prevent storing the incorrect result.

Applications

- Digital signal processing
- Embedded systems
- Financial software
- Scientific calculations

Conclusion

By detecting overflow using the overflow flag or carry comparison, the processor can identify incorrect results. Using larger data types or overflow handling techniques ensures accurate signed arithmetic operations.

2. Debugging Ripple Carry Adder Delay

Introduction

A Ripple Carry Adder (RCA) is a simple circuit used for binary addition. In this adder, the carry generated by one stage must propagate to the next stage, which increases the computation time. This delay becomes significant in high-speed and real-time systems.

Problem

A real-time processor using a Ripple Carry Adder (RCA) executes arithmetic operations slowly.

Cause of Delay

In an RCA, the carry must travel through every bit.

Example (4-bit):

Bit0 → Bit1 → Bit2 → Bit3

Each full adder waits for the previous carry.

Total delay:

Delay = Number of bits × Full Adder delay

For larger adders (16-bit, 32-bit, 64-bit), the delay becomes significant.

Performance Bottleneck

- Carry propagates sequentially.
- Higher bits cannot compute until lower bits finish.
- This increases execution time.

Optimized Solution: Carry Look-Ahead Adder (CLA)

CLA calculates carry signals in parallel using:

Generate:

$$G = A \times B$$

Propagate:

$$P = A \oplus B$$

Carry:

$$C1 = G0 + P0C0$$

All carries are generated simultaneously.

Advantages

- Much faster addition.
- Parallel carry computation.
- Lower propagation delay.
- Better performance for high-speed processors.

Applications

- Modern CPUs
- DSP processors
- GPUs
- Real-time embedded systems

Conclusion

Replacing the Ripple Carry Adder with a Carry Look-Ahead Adder (CLA) greatly reduces carry propagation delay by generating carries in parallel. This improves processor speed and overall system performance.

3. Debugging Booth's Multiplication Algorithm

Introduction

Booth's algorithm is an efficient method for performing signed binary multiplication. It reduces the number of addition and subtraction operations by examining pairs of bits. Incorrect implementation of bit checking or shifting can lead to wrong multiplication results.

Problem

A multiplication unit using Booth's algorithm gives incorrect answers for some negative numbers.

Areas to Check

1. Bit-Pair Checking

Booth's algorithm examines two bits:

Q0 Q-1

Rules:

- 00 → No operation
- 01 → Add multiplicand
- 10 → Subtract multiplicand
- 11 → No operation

Incorrect bit checking produces wrong multiplication.

2. Arithmetic Right Shift

After every operation:

- Shift A, Q, and Q-1 together.
- Preserve the sign bit.

Using a logical shift instead of an arithmetic shift changes the sign incorrectly.

3. Sign Extension

When shifting:

- The MSB must be copied.
- Negative numbers must remain negative.

Failure to sign-extend causes incorrect results.

Possible Errors

- Incorrect bit-pair interpretation.
- Logical shift used instead of arithmetic shift.
- Missing sign extension.
- Incorrect iteration count.
- Wrong initialization of Q-1.

Suggested Corrections

- Verify Q0 and Q-1 before each step.
- Use arithmetic right shifts.
- Preserve the sign bit.
- Perform the correct number of iterations.
- Test using positive and negative operands.

Applications

- Computer processors
- Digital signal processors
- ALUs
- Embedded controllers

Conclusion

Proper bit-pair checking, arithmetic right shifting, and correct sign extension ensure accurate signed multiplication. Correct implementation of Booth's algorithm improves both accuracy and hardware efficiency.

4. Debugging Restoring Division Algorithm

Introduction

The restoring division algorithm is a binary division technique used in computer processors. It performs repeated subtraction and restoration operations to determine the quotient and remainder. Frequent restoration steps increase execution time and reduce efficiency.

Problem

An embedded processor using restoring division is slow because restoration occurs frequently.

Workflow

Steps:

1. Shift dividend.
2. Subtract divisor.
3. If result is negative:
 - Restore previous value.
 - Set quotient bit to 0.
4. Repeat.

Source of Inefficiency

Whenever subtraction gives a negative value:

- The algorithm restores the original value.
- This introduces an extra addition operation.

Frequent restoration increases:

- Execution time
- Power consumption
- Hardware operations

Optimized Solution: Non-Restoring Division

Instead of restoring immediately:

- If subtraction is negative, continue with an addition in the next cycle.
- Restoration is avoided after every failed subtraction.

Advantages

- Fewer arithmetic operations.
- Faster execution.
- Reduced hardware complexity.
- Better processor efficiency.

Applications

- Embedded processors
- Microcontrollers
- Digital signal processing
- High-speed arithmetic units

Conclusion

Using the non-restoring division algorithm eliminates unnecessary restoration operations, reducing computation time and improving processor performance. This method is more suitable for high-speed arithmetic operations.

5. Debugging IEEE 754 Floating-Point Addition

Introduction

IEEE 754 floating-point representation is the standard method used to store and process real numbers in computers. During floating-point addition, exponent alignment, normalization, and rounding may introduce precision errors, especially when numbers with very different magnitudes are added.

Problem

A scientific application produces inaccurate results when adding a very small number to a very large number.

Example:

$$1.0 \times 10^{10} + 1.0$$

Causes of Inaccuracy

1. Exponent Alignment

Before addition:

- Smaller number's mantissa is shifted right.
- Significant digits are lost.

This loss reduces precision.

2. Normalization

After addition:

- Result is normalized.
- Additional bits may be discarded.

This can introduce small errors.

3. Rounding Errors

IEEE 754 stores only a limited number of mantissa bits (23 bits in single precision).

Extra bits are rounded, causing precision loss.

Optimization Techniques

Use Double Precision

- Single precision: 32 bits
- Double precision: 64 bits

Double precision provides:

- More exponent bits
- More mantissa bits
- Higher accuracy

Algorithmic Improvements

- Add numbers of similar magnitude first.
- Use compensated summation methods (such as Kahan summation) to reduce accumulated rounding error.
- Avoid unnecessary repeated floating-point operations.

Advantages

- Improved numerical accuracy.
- Reduced rounding errors.
- Better precision for scientific and engineering applications.

Applications

- Scientific computing
- Machine learning
- Weather forecasting
- Financial analysis
- Engineering simulations

Conclusion

Using double-precision floating-point representation and improved summation algorithms reduces rounding errors and increases numerical accuracy. These techniques provide more reliable results in scientific, engineering, and financial applications.

Code of Conduct:

I N. Tharun Sai / 192472382 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

MARKS ALLOCATION

	Identification of Errors / Bugs (20%)	Debugging Approach & Analysis (20%)	Correctness of Debugged Solution (25%)	Optimization Techniques Applied (20%)	Testing and Documentation (15%)
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Identification of Errors / Bugs	20%	Accurately identifies all errors and issues with complete understanding of the problem	Identifies most errors with minor omissions	Identifies limited errors; misses key issues	Unable to identify errors or misunderstands the problem
Debugging Approach & Analysis	20%	Uses effective debugging techniques with clear analysis and root cause identification	Applies suitable debugging methods with minor gaps	Uses basic debugging methods with limited analysis	No systematic debugging approach followed
Correctness of Debugged Solution	25%	Provides completely corrected solution with accurate functionality and expected output	Provides working solution with minor errors	Partially correct solution with limited functionality	Incorrect solution or fails to resolve errors
Optimization Techniques Applied	20%	Applies efficient optimization methods to	Performs optimization with	Limited optimization with minimal improvements	No optimization applied or

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
		improve performance, memory usage and quality	noticeable improvements		inefficient implementation
Testing and Documentation	15%	Performs complete testing with proper validation and clear documentation	Provides adequate testing and documentation with minor issues	Limited testing and incomplete documentation	No proper testing or documentation provided